

Slide 1: The `grun` Philosophy — Democratizing Exascale

The Problem: The "Heroic Effort" Gap

- Ideal static GPU grids assume data is already perfectly staged in device memory.
- In reality, scientists rely on synchronous `hipMemcpy` staging, leaving GPUs starved for data.
- Closing this gap requires "heroic" software engineering: async streams, pinned memory, manual I/O overlap.
- The result: For many streaming and multi-stage scientific workflows, the dominant bottleneck is data movement and staging rather than arithmetic throughput.

The `grun` Solution (A Physics-Based Approach)

- `grun` is not an incremental CS tweak; it is derived from first-principles dataflow invariants (conservation of mass, locality, monotonic time). It absorbs the nanosecond overhead of atomic claiming to eliminate the millisecond stalls of synchronous data transfers.
- **The Contract:** The domain scientist writes a single-wavefront compute kernel. `grun` owns the async staging, NUMA-aware routing, dynamic load-balancing, and persistent execution grid.

Slide 2: STTR Phasing & Execution Strategy

Extracting our shell-coupled parallelizer into a C-library and mapping it to bare-metal GPUs requires deliberate sequencing. **We seek co-PIs to help bridge the gap between our theoretical dataflow models and bare-metal ROCm realities.**

Phase I: `libforkrun` + Sledgehammer Runtime (The Commercial Substrate)

- **Deliverable:** Production-quality asynchronous CPU→GPU staging engine.
- **Scope:** Thread-safe `memfd` substrate, CPU SIMD scanning, NUMA-aware placement, overlapped DMA.
- **Success Criterion:** Existing multi-stage HIP applications run unchanged and see 2x-3x speedups purely from staging automation. A low-risk, high-ROI facility solution.

Phase II: Fused Persistent-Workgroup Engine (The Joint Research Scope)

- **Deliverable:** Bare-metal persistent workgroups with LDS-bounded modulo routing.
- **Scope:** Mapping hierarchical claiming, Circulating Permit arenas, and "breathing-grid" autotuning to AMD CDNA architectures.
- **The Fallback Contract:** *Any fused segment may be abandoned at runtime and safely replayed through the Phase-I Sledgehammer path.*

Slide 3: Competitive Landscape & Positioning

The State of the Art:

- **NVIDIA DALI / AMD rocAL:** GPU-accelerated data loading specifically for ML training. *Limitation:* Fixed pipeline stages; cannot express arbitrary multi-stage scientific pipelines.
- **HIP / CUDA Graphs:** Amortize kernel launch overhead to single-digit microseconds. *Limitation:* Still relies on static scheduling; cannot dynamically load-balance irregular data.
- **Legion / Regent:** Data-driven task parallelism via logical regions. *Limitation:* Heavyweight runtime overhead; designed for multi-node task graphs, not single-node, high-throughput streaming filters.

The `grun` Niche: `grun` abstracts the *data pipeline around the kernel*. It is an intra-node, bare-metal framework designed specifically for Frontier-class AMD MI250X/MI300X architectures, utilizing contention-free permit circulation and born-local XCD placement.

Slide 4: The Persistent Workgroup Model

HIP and CUDA Graphs amortize kernel launch latency but fail to solve the structural bottlenecks of streaming pipelines. `grun` proposes solving this via **Persistent Workgroups**:

1. **Silicon-Bounded Launch:** Instead of over-subscribing the GPU and relying on the hardware scheduler (which risks deadlock), `grun` launches exactly `Num_CUs × Max_Active_Blocks`. The grid physically fills the silicon.
2. **Zero Inter-Batch Overhead:** Workgroups loop internally, dynamically drawing macro-blocks of data via HBM atomics. Transitions bypass the host driver entirely.
3. **Intra-Workgroup Data Fusion:** Persistent workgroups allow a single SM/CU to compute Stage 1, synchronize via `s_barrier`, and immediately compute Stage 2 on the exact same data. Separate kernel launches force an HBM read/write round-trip.

Slide 5: Architectural Lineage (Proven CPU Substrate → GPU Research)

Our theoretical models are grounded in empirical success. The `forkrun` CPU architecture already achieves 200x speedups over GNU Parallel by enforcing these physical invariants. `grun` investigates whether these invariants can be extended to AMD CDNA architectures.

Substrate-Independent Principle	Demonstrated in <code>forkrun</code> (CPU)	Proposed GPU Mechanism (<code>grun</code>)
1. Upstream pre-computes bounds	SIMD Scanner parses delimiters	CPU DMA batcher pre-aligns slabs
2. Pull-based claiming	<code>atomic_fetch_add</code> (Cache coherent)	Hierarchical Dispensing (HBM → LDS)
3. Born-local placement	<code>set_mempolicy(MPOL_BIND)</code>	Direct-to-XCD explicit HBM Pools
4. Escrow outside the fast path	Priority-inversion OS pipe	Workgroup-Aggregated Doorbell

Conclusion: The design explores enforcing monotonic forward progress and data locality without relying on reactive thread-locking.

Slide 6: Scope Boundary — Intra-Node Specialization

`forkrun` / `grun` own the intra-node stack, end-to-end:

- CPU-side preprocessing via `libforkrun` (parsing, filtering, transformation).
- Born-local NUMA → born-local XCD/GCD placement (continuous data locality).
- CPU→GPU async DMA staging, persistent workgroup management, variance recovery.

Inter-node coordination is explicitly out of scope:

- **SLURM** handles node allocation and launch.
- **MPI** (or per-node file partitioning) handles data distribution across nodes.
- Each node runs one independent `grun` instance per physical GPU against its local data slice.

Why this boundary: Single-node data pipeline optimization is a discrete, fundable problem with massive ROI. We integrate seamlessly with existing distributed facility schedulers.

Slide 7: End-to-End Born-Local Pipeline (Zero CPU Extra Copies)

One Substrate, Three Placement Decisions:

1. **Ingest `memfd`** : Born-local to NUMA node via `MPOL_BIND ( libforkrun )`.
2. **Global Output `memfd`** : CPU-side aggregation, registered via `hipHostRegister` for DMA sourcing.
3. **XCD-Local HBM Arenas**: Born-local via direct Infinity Fabric targeting (`grun`).

Single-Hop Data Path:

The CPU SIMD batcher scans the global `memfd` in place and emits `{offset, length}` descriptors. The DMA engine reads directly from the registered pages and explicitly targets single contiguous HBM buffers per XCD (zero pointer chasing). Data moves exactly once, and the Infinity Fabric remains perfectly quiet during compute.

Slide 8: The Segment-Local Invariant: The Immortal Record

The `grun` Solution: 1 Input = 1 Immortal Logical Record

- `grun` decouples the *control plane* (record count) from the *data plane* (byte size). If a kernel filters a record, it emits a metadata pointer with `length = 0`. The record count is conserved. The user statically declares `max_record_bytes`; outlier records hit Escrow.
- **Why this is the load-bearing spine:** Record conservation is what makes stateless modulo routing, hardware-scatter output ordering, and fused execution mathematically viable.

The Downstream Ownership Transition Invariant:

- Memory footprint transitions dynamically to Stage N 's budget the microsecond it is published to the $N - 1 \rightarrow N$ ring.
- This provides instantaneous backpressure feedback, shutting the CPU admission gate before a hard OOM occurs on the device.

Slide 9: Execution Paths & The Two Scheduling Domains

`grun` offers two execution paths. Sledgehammer is the commercial foundation; the Fused Engine is the proposed research accelerator.

Path 1: Sledgehammer (Phase I Foundation)

- **Use Case:** Workloads with extreme VGPR discontinuity or massive data amplification.
- **Mechanism:** Unfused kernel launches. Data returns to host memory between stages for CPU rebatching. Acts as a safety net and general-purpose async staging engine.

Path 2: The Fused Engine (Phase II Accelerator)

- The Fused engine provides **two independent scheduling domains**, and all behaviors emerge from how work is partitioned between them:
 1. **Runtime-Managed Domain (Immortal Records):** Claimed, routed, and ordered by the runtime via modulo math. Cannot be created/destroyed.
 2. **Kernel-Managed Domain (Subrecords):** Live inside records. Created/destroyed/filtered by the user's kernel. Invisible to the runtime scheduler.

Slide 10: Hierarchical Claiming & LDS-Bounded Modulo Routing

1. Hierarchical Dispensing: Explicit two-level dispensing: Global HBM Ring $\rightarrow$ LDS Broadcast $\rightarrow$ LDS Atomic Claim (`ds_add_rtn_u32`). Eliminates HBM hotspots.

2. The Hardware Gate: The `s_barrier` acts as a hard hardware gate: Stage 2 wavefronts cannot claim LDS survivors until Stage 1 completely finishes writing them.

3. The Modulo Routing Math (Intra-Workgroup):

Wavefronts execute Stage 2 based on their share of N survivors across M wavefronts:

$$\lfloor N/M \rfloor + \begin{cases} 1 & \text{if } ((wID + arena_id) \bmod M) < (N \bmod M) \\ 0 & \text{otherwise} \end{cases}$$

- **The Global Invariant:** `∀ cycles: sum(wave_allocations) == N_survivors` . (e.g., 10 survivors across 8 waves. Waves 0-1 process 2 items; Waves 2-7 process 1 item. Sum = 10).
- **Arena-Based Rotation:** Offsetting the modulo math by `arena_id % M` decorrelates persistent wave identity from long-run scheduling bias.

Slide 11: The Unified Fused Architecture (One Equation of State)

There are no separate "modes" in the Fused engine. `grun` executes a **single, unified codebase**. All execution behaviors emerge from how the user configures one parameter: the ratio of Records to Subrecords.

Configuration Slider	Runtime-Managed (Records)	Kernel-Managed (Subrecords)	Emergent Phase / Behavior
N Records, 1 Subrecord	Active	Inactive	"Chisel" (Uniform Workloads): Every item is tracked by the runtime. The modulo math triggers LDS data compaction, perfectly balancing survivor records across the workgroup.
1 Record, N Subrecords	Inactive	Active	"Conveyor" (High-Selectivity): Runtime tracks 1 pointer. Wavefronts loop heavily over subrecords in L1. Empty items cost zero routing overhead. Best for 99% filter rates.
N Records, M Subrecords	Active	Active	"Hybrid": The math natively balances L1 cache pressure (Subrecords) against LDS routing compaction (Records).

The Fallback Contract: *If data variance completely breaks uniformity or user contracts are violated, the Fused engine is abandoned and data falls back to the unfused **Sledgehammer** path.*

Slide 12: Defeating Deadlocks (The Correctness Invariant)

Mid-kernel `malloc` / `free` or classic CAS retry loops risk hardware livelocks. `grun` secures memory structurally via a proposed **Circulating Permit System**.

The Core Invariant: *At all times, in-flight execution is strictly upper-bounded by circulating permits.*

HBM Circulating Permit Channel

- A "Token" does not represent absolute memory allocation; it represents **logical exclusivity over a bounded arena region for a single cycle**.
- **The Draw:** A starting wave executes `read_index = atomic_fetch_add(&read_idx)`, then spins on `load_acquire` with an `s_sleep` backoff. *Zero CAS contention, because the slot is uniquely owned.*
- **The Return:** A finishing wave executes `write_index = atomic_fetch_add(&write_idx)`, then publishes the token via `store_release`.
- **Result:** A bounded baton-passing system. The producer is strictly bounded by the fixed token population (K_{max}), mathematically preventing lap/overwrite races.

Slide 13: The "Breathing Grid" (Active Occupancy Throttling)

While Slide 12 guarantees structural correctness, the Breathing Grid represents the dynamic **control policy layer**, throttling token issuance to optimize utilization.

The Theory: Removing Active Scheduler Pressure

- **The Dynamic Population:** $K_{current}$ acts as a Throttle Semaphore, dictating how many permits are allowed to circulate.
- **Resource Release:** When $K_{current}$ shrinks, dormant waves enter a low-frequency sleep loop. This reduces active scheduling, instruction cache, and memory bandwidth pressure, potentially yielding SM resources for concurrent streams (like Sledgehammer fallbacks). *Validating this physical behavior on AMD CDNA schedulers is a key STTR research goal.*
- **Look-Ahead Memory Condition:**

$$\max(\text{ActualMemory}_N, \text{InFlight}_{N-1} \times \text{ModeledMemoryPerWave}_N) < \text{Budget}_N$$

Slide 14: Model Bootstrapping & Global HBM Decay

Model Bootstrapping & Decimation Timing:

- Memory model updates are evaluated on a dynamic CPU scheduler:

$$\text{Interval} = wID \bmod (\min(16 \cdot \text{fast_log2}(wID + 1), 128)) \quad \text{OR } 100 \text{ ms}$$

- Enforces a hyper-conservative start ($K_{\text{start}} = 10$) to safely capture empirical scaling properties before ramping up the grid.

Multi-XCD Local/Global Decay:

- To prevent localized OOM on multi-die chips (MI300X), decay is triggered if:

$$(\text{Local HBM} > 90\% \text{ AND Total GPU HBM} > 90\%) \text{ OR } (\text{Local HBM} > 95\%)$$

- **Proportional Decay with Cooldown:**

$$K_{t+1} = K_t \times \left(1 - \beta \times \frac{\text{Local HBM} - 0.90}{0.10} \right)$$

Followed by a lockout cooldown. Decay writes to the atomic K_{current} throttle; it does not touch the K_{max} ring capacity.

Slide 15: Logical Fault Resilience & Variance Escrow

The Principle: State goes where state is cheap. Compute goes where compute is fast. Bare-metal ALUs cannot elegantly handle massive data spikes. If a freak stochastic batch breaches its Arena limits, the GPU remains stateless and punts to the CPU.

Workgroup-Aggregated Escrow (Defeating PCIe Storms):

- **Tier 1 (Stochastic Spike / NaN / Arena Overflow):** If data variance causes a macro-block to exceed Arena limits, the Workgroup aborts the block. Using `__ballot_sync` (and `s_barrier`), the failure is coalesced. A single lane rings the host-pinned doorbell **exactly once per batch** (a single PCIe payload). *The Workgroup returns its Arena Permit to the channel, rings the doorbell, and draws fresh work.*
- **The CPU Re-batch (The Fallback Contract):** The CPU dispatcher reads the doorbell, references the pristine `memfd` staging ground, and routes the outlier batch dynamically into the unfused Sledgehammer pipeline.

Slide 16: Proposed Joint Research Scope (STTR Phase II)

To realize this theoretical model, we seek HPC Systems co-PIs to solve the following bare-metal hardware mapping challenges:

#	Research Challenge	Proposed Mitigation / Joint Investigation
R1	Linux TDR Watchdogs	Mitigating OS timeouts for persistent I/O-bound wavefronts (driver-level heartbeats or context-yielding primitives).
R2	Bare-Metal XCD Topology	Exposing Infinity Fabric/XCD topology constraints directly to user-space <code>hipMalloc</code> routing for true born-local placement.
R3	LLVM VGPR Fusion Limits	Fused max-VGPR assumptions are often violated by LLVM. Investigate building a custom compiler pass (<code>gruncc</code>) for strict register scoping.
R4	Hardware Livelock Avoidance	Validating the Circulating Permit <code>load_acquire</code> loop against AMD CDNA scheduler fairness guarantees.
R5	Memory Visibility in Fusion	Validating <code>s_barrier</code> hardware guarantees and LDS data compaction across stages.
R6	PCIe Doorbell Flooding	Verifying workgroup-level coalescing prevents host-bus (PCIe) saturation during extreme data variance spikes.

Slide 17: User Contract & Tier 0 Guardrails

To guarantee exascale performance, `grun` enforces a strict boundary of responsibility. Violating the contract results in a safe fallback to Sledgehammer, not a system crash.

Responsibility	Domain Scientist (The Compute)	Enforced By (<code>grun</code> Alpha Guardrails)
1. Memory Limits	Keep allocations within K_{max} capacity.	Launch Canary: <code>sharedSizeBytes <= HW_Limit</code> . (Hard Check)
2. Uniform Control	All active paths must hit <code>grun_stage_barrier()</code> .	Compiler Pass: Static LLVM analysis of barrier uniformity. (Hard Check)
3. VGPR Footprint	Isolate variables to distinct <code>if/else</code> branches.	Diagnostic Canary: Host warns if <code>numRegsPerThread</code> > Expected.
4. Arena Integrity	Do not manually touch memory pointers/pools.	API Opacity: <code>grun_arena_t</code> is an opaque handle.
5. Forward Progress	Kernels must be mathematically guaranteed to terminate.	User Contract: Documented requirement (Halting Problem limits static checks).

Slide 18: Hypothesized Stochastic Filter Breakthrough

The Bottleneck: High-selectivity stochastic filters (e.g., 1% pass-through rate) force experts to choose between three structural taxes:

1. **The Divergence Tax:** Run the kernel anyway (Effective throughput: ~1%).
2. **The Compaction Tax:** Global GPU compaction via `cub::DeviceSelect` (~40-50% effective throughput due to global memory barriers).
3. **The Interconnect Tax:** CPU pre-filtering (~30% effective throughput).

Proposed `grun` Solution: The Unified Fused Engine

- **Unmodified Kernels:** We project `grun`'s async staging will yield a $\geq 30\%$ **speedup** over naïve SOTA with zero user effort via Sledgehammer.
- **Chisel Phase:** Fused with 1 record = 1 input. LDS Modulo math compacts survivors natively in shared memory. Projected to bypass HBM entirely for 20-50% speedups over `cub::DeviceSelect`.
- **Conveyor Phase:** Runtime manages 1 Record; kernel loops subrecords. Expected to excel under extreme data variance, converting filter rates into control-plane stability.

Slide 19: Evaluation Plan & Target Hardware

Target Hardware: Validation proposed on OLCF Frontier (MI250X) and/or MI300X testbed systems to test true exascale topologies.

The Dual Baselines:

1. **Baseline A (The "Typical Scientist"):** Synchronous `hipMemcpy`, blocking host launches, no I/O overlap.
2. **Baseline B1 (Global Compaction):** Hand-tuned `cub::DeviceSelect`.
3. **Baseline B2 (In-Place / Custom):** Hand-tuned `thrust::remove_if` / Custom stream compaction.

Proposed Quantitative Targets:

Workload Skew	Success (vs Typical)	Target (vs Typical / vs Expert)	Stretch (vs Typical / vs Expert)
W0 (Balanced)	+ 10%	+ 30–40% / within 10%	+ 40–60% / tie
W1 (Mild skew)	+ 15%	+ 40–60% / + 5–10%	+ 60–80% / + 10–20%
W2 (Heavy skew)	+ 25%	2× / + 10–20%	3× / + 20–30%
W2E (Extreme/1% Pass)	2×	10× / + 20–30%	20× / + 30–50%

Slide 20: The Dual Value Proposition (STTR Impact)

`grun` proposes solving a massive facility-wide usability problem while exploring fundamentally new paradigms in GPU systems research.

1. The Economic Value (Democratizing Exascale)

- **The Impact:** Automating expert-level async data staging eliminates "heroic" software engineering. 10-100% GPU utilization improvements at ~\$1M/day operational cost equals tens of millions annually.
- **Licensing Model:** STTR IP retention applies. Core Runtime (`grun` / `libforkrun`) is Apache 2.0. Free for Government (CRADA) and Academic use. Commercial entities running on private hardware require Enterprise licenses.

2. The Academic Value (Advancing the State of the Art)

- **The Impact:** The project explores whether conservation-law-inspired dataflow models can be successfully applied to GPU runtime design. Our theoretical models—Unified Fused Equation of State, Circulating Permit Arenas, and VGPR-Aware Flow Control—represent a potentially new class of GPU runtime abstractions. We need your expertise to map them to silicon.

Appendix: Kernel Annotation Spec (Alpha Draft)

To support `gruncc` compilation and Launch Canaries, domain scientists decorate their kernels with C++11 attributes.

```
// [[grun::vgpr_max(128)]] declares the contract limit for the kernel
// [[grun::io(...)]] enables auto arena sizing and struct reflection
[[grun::vgpr_max(128)]]
[[grun::io(in=sizeof(Particle), out=sizeof(Track), expansion=1.5)]]
__global__ void my_fused_pipeline(grun_arena_t arena, const Input* in) {

    // Global ordering is derived from the HBM draw, not just LDS-local wID:
    // ingress_id = macro_block_base + wID
    uint64_t ingress_id = grun_get_ingress_id();

    // Uniform control flow enforced by LLVM Pass
    [[grun::stage(1)]]
    process_stage_1(arena, in, ingress_id);

    grun_stage_barrier(); // Hardware gate

    [[grun::stage(2)]]
    process_stage_2(arena);
}
```